AegisX – AI-Powered Cybersecurity Assessment Platform

Development Log & Software Changelog

1. Project Information & Timeline

- **Project Name:** AegisX – AI-Powered Cybersecurity Assessment Platform
- **Repository:** github.com/username/AegisX
- **Lead Architect & Developer:** Engineering Lead
- **Project Status:** Phase 1 Development (v0.1.0 to v1.0.0)

Plaintext

```
+-------------------------------------------------------------------------------+
|                          AEGISX MILESTONE TIMELINE                            |
+-------------------------------------------------------------------------------+
| Week 1 (v0.1.0 - v0.2.0): Core Infrastructure, Scanning Engines, & Validation |
| Week 2 (v0.3.0 - v0.4.0): Risk Engine, SSL/TLS, Headers, & DNS Modules        |
| Week 3 (v0.5.0 - v0.8.0): Streamlit UI, SQLite Persistence, & ReportLab PDF Engine|
| Week 4 (v0.9.0 - v1.0.0): System Integration, Hardening, Testing, & v1.0 Release |
+-------------------------------------------------------------------------------+
```

2. Git Commit & Branching Guidelines

To maintain clean repository management, standard **Conventional Commits** and **Git Flow** specifications are enforced.

Commit Syntax Strategy

Plaintext

```
<type>(<scope>): <short summary>

[optional body]
```

- **Types:**
  - feat: A new end-user feature.
  - fix: A bug fix in existing logic.
  - refactor: Code restructures that neither fix bugs nor add features.
  - docs: Documentation updates only (e.g., README, PRD, SRS, SDD).
  - test: Adding missing tests or refactoring test suites.
  - chore: Build tasks, dependency updates (requirements.txt), or config changes.
- **Examples:**
  - feat(port_scanner): implement async TCP socket worker pool
  - fix(validator): resolve regex bypass on domain input sanitization
  - docs(srs): add IEEE 830 functional requirements section

3. Sprint Planning & Task Tracker

Sprint ID	Milestone	Focus Area	Status	Target Version
Sprint 1	Foundation	Project setup, Input Validation, Asynchronous TCP Scanner, Banner Grabber	**Completed**	v0.1.0 - v0.2.0
Sprint 2	Security Engines	HTTP Header Analyzer, SSL/TLS Inspection, DNS/WHOIS Recon, Risk Engine	**Completed**	v0.3.0 - v0.4.0
Sprint 3	UI & Persistence	Streamlit Dashboard, SQLite DAO, Plotly Gauges, PDF & CSV Exporters	**Completed**	v0.5.0 - v0.8.0
Sprint 4	Release Hardening	End-to-End Integration, Input Hardening, Unit Tests, Final Release Docs	**Completed**	v0.9.0 - v1.0.0

4. Detailed Changelog (Semantic Versioning)

[1.0.0] - Baseline MVP Release

Release Date: Target MVP Release

Status: Production Ready

Added

- **Final Release Documentation:** Included finalized IEEE 830 SRS, System Design Document (SDD), and Product Requirements Document (PRD).
- **Input Hardening:** Implemented strict regex sanitization in core/validator.py to prevent command injection risks.
- **End-to-End Integration:** Tied Streamlit frontend directly to core scan orchestration loops and PDF export pipelines.

Fixed

- Handled socket connection dropouts gracefully when scanning firewalled targets to prevent UI thread hangs.

[0.8.0] - ReportLab PDF & CSV Export Engines

Release Date: Sprint 3 - Day 12 to 14

Added

- **PDF Exporter:** Integrated reports/pdf_generator.py using ReportLab to compile clean executive summaries.
- **CSV Exporter:** Implemented raw data extraction using pandas for audit dataset downloads (reports/csv_generator.py).
- **Download Buttons:** Added native download triggers in the Streamlit action panel.

[0.6.0] - SQLite Database Integration

Release Date: Sprint 3 - Day 8 to 10

Added

- **Database Schema:** Built db/database.py using sqlite3 to persist target metadata, scan runs, and individual finding records.
- **Persistence Operations:** Implemented parameterized CRUD functions to safely log scan results without SQL injection vulnerabilities.

[0.5.0] - Interactive Streamlit UI Dashboard

Release Date: Sprint 3 - Day 6 to 7

Added

- **Dashboard Interface:** Built single-page interactive interface using Streamlit (app.py).
- **Plotly Visualizations:** Integrated dynamic risk gauges and metric summary cards.
- **Tabbed Navigation:** Structured result outputs into clean tabs (Network, Headers, SSL, Recon, Recommendations).

[0.4.0] - Risk Engine & Heuristic Calculation

Release Date: Sprint 2 - Day 4 to 5

Added

- **Scoring Logic:** Implemented core/risk_engine.py using a weighted rule model to generate normalized ratings ($0.0 - 10.0$).
- **Remediation Mapping:** Built a lookup dictionary linking detected vulnerabilities directly to step-by-step remediation guidance.

[0.3.0] - Security & Recon Modules

Release Date: Sprint 2 - Day 1 to 3

Added

- **HTTP Header Analyzer:** Created core/header_analyzer.py to audit key response headers (HSTS, CSP, X-Frame-Options).
- **SSL/TLS Inspector:** Built core/ssl_inspector.py to evaluate server certificate validity, expiration dates, and cipher suites.
- **DNS & WHOIS Module:** Integrated dnspython and python-whois in core/recon.py to fetch domain ownership records.

[0.2.0] - Async Port Scanner & Banner Grabber

Release Date: Sprint 1 - Day 3 to 5

Added

- **Asynchronous Scanner:** Implemented core/port_scanner.py using Python asyncio loop pools for rapid multi-port connect checks.
- **Banner Extraction:** Built handshake parsing logic to capture raw server greeting banners on open TCP ports.

[0.1.0] - Initial Repository Setup

Release Date: Sprint 1 - Day 1 to 2

Added

- **Project Structure:** Created initial modular directory layout (core/, ui/, db/, reports/, tests/).
- **Input Validator:** Built core/validator.py with regular expressions to validate domain names and IPv4/IPv6 address strings.
- **Environment Setup:** Configured requirements.txt with external dependencies (streamlit, requests, reportlab, plotly, dnspython, python-whois).

5. Technical Decision Records (TDR)

TDR-01: Native Python Sockets vs. External Nmap Binary

- **Context:** The port scanner module required a mechanism to identify open TCP ports.
- **Decision:** Selected native Python socket and asyncio libraries over wrapping external nmap system binaries.
- **Rationale:** Eliminates host-level CLI dependencies, ensuring cross-platform execution on Linux, macOS, and Windows without root privilege restrictions.

TDR-02: ReportLab for PDF Compilation

- **Context:** The application required an automated method to output printable executive summaries.
- **Decision:** Adopted reportlab canvas and flowable elements.
- **Rationale:** Provides strict layout control and fast compilation times ($\le 3 \text{ seconds}$) without requiring headless web browser rendering engines (e.g., Puppeteer, wkhtmltopdf).

6. Daily Progress Logs (Sample Template)

Plaintext

[YYYY-MM-DD] - Day X Log

Task Assigned : Build SSL/TLS certificate inspection parser (FR-SSL-01, FR-SSL-02)
Work Completed : Created core/ssl_inspector.py using Python ssl module socket context.
                 Extracted issuer, valid date ranges, and protocol versions.
Blockers/Issues : Handling self-signed certificates caused SSL verification errors.
Resolution   : Implemented ssl.CERT_NONE fallback mode specifically for metadata
                 extraction while flagging untrusted chain status in findings.
Commit Hash   : a1b2c3d (feat(ssl): implement cert chain parsing and fallback)
---

7. Known Issues & Remediation Backlog

Issue ID	Module	Description	Severity	Workaround / Planned Fix
ISS-01	recon.py	Rate limiting on WHOIS port 43 requests when performing frequent sequential lookups.	Medium	Add cached lookup fallbacks in SQLite database layer.
ISS-02	port_scanner	Stealth firewalls dropping packets silently cause connection worker timeouts to hit maximum limits.	Low	Tune default socket connection timeout to $\le 1.0 \text{ second}$ in config.py.

8. Future Roadmap Items (v2.0.0 Backlog)

- **AI Threat Explanations:** Integrate LLM API calls (OpenAI / Gemini) to contextually explain risks and offer platform-specific config fixes.
- **NIST NVD CVE Integration:** Query public vulnerability databases automatically using service banner strings.
- **Threat Intelligence Feeds:** Query external IP threat APIs (VirusTotal, AbuseIPDB, Shodan).
- **Scheduled Scans & Alerts:** Implement background cron tasks with email notifications for automated monitoring.